HP-F2: Applied Tumor Immunity

Week 3: Bioinformatics

Differential gene expression analysis using DESeq2

December 1st – 5th, 2025
Supervisor: Dr. Cihan Erkut and colleagues

Yilu Wang

Master Student in Molecular Biosciences (Major Cancer Biology)
Heidelberg University and DKFZ

Abstract

In this practical, we analyzed Illumina RNA-seq data from UM-Chor1 chordoma cells expressing either the TBXT-targeting DARPin D4 or NTC, each in biological triplicate. Differential expression analysis was performed using DESeq2[1]. Gene-level read count data were imported into R, and active genes were selected with a zFPKM cutoff of −3 according to TPM values. Following batch effect detection, we modelled the differential expression of D4 compared to NTC using DESeq2. Empirical Bayes shrinkage was then used to adjust genes with low counts or high dispersion. Genes with a more than 1.5-fold change and an FDR less than 1 were considered significantly regulated. Finally, we performed functional enrichment analysis with Enrichr to identify biological patterns associated with TBXT inhibition in chordoma cells.

Introduction

Dataset description

In this report, we analyze a subset of the Umbaugh et al. RNA-seq dataset from UM-Chor1 chordoma cells engineered to express either the TBXT-targeting DARPin D4 or the non-targeting control DARPin E3_5 (NTC)[2], in order to assess transcriptional consequences of TBXT inhibition. For each condition, three independent biological replicates were used. RNA was extracted and sequenced on an Illumina HiSeq 4000 platform.

Illumina RNA-seq and data preprocessing

Illumina RNA-seq first converts RNA into cDNA fragments and ligated them with adapters, which serve as primer handles for amplification. Then, the sequencing library is hybridized to a flow cell. Fragments are then extended with fluorescent nucleotides for imaging. This process is called sequencing by synthesis. The preliminary output is a FASTQ file containing reads and corresponding quality scores, which is then aligned to reference genomes and summarized into a gene-specific count matrix for downstream analysis.

Differential expression analysis

DESeq2 workflow. DESeq2 inputs a matrix of raw gene counts and sample metadata, constructs an object with a design formula, estimates size factors and dispersions, fits negative binomial generalized linear models (GLMs) per gene, performs Wald tests with false discovery rate (FDR) correction, and shrinks log2 fold changes (LFCs) for stable effect size estimates and visualization. Results are called by setting thresholds for FDR and LFC.

zFPKM-based filtering. Active genes are selected from background using zFPKM normalization[3]. We apply this method to transcript-per-million (TPM) values, which are corrected for gene length and per-sample library size, as opposed to raw counts. TPMs of all genes are log-transformed and standardized relative to the Gaussian fit of the per-sample density plot, and genes with the zFPKM < -3 are filtered out.

Batch effect detection. Batch effects are assessed by PCA on rlog-transformed counts. Batch correction methods such as ComBat are useful for visualization, but for differential expression analysis the strategy is to include batch as a term in the design so uncertainty is modeled.

Dispersion shrinkage. The main advantage of DESeq2 is that it uses shrinkage estimation for dispersions (and fold changes) to improve stability and interpretability of estimates. Gene dispersion

(or equivalently, variance) is estimated per gene and then shrunken gene-wise toward a fitted mean–dispersion trend over all samples. This process is important because overdispersion, meaning the variance exceeds the mean, is common in RNA-seq data, especially when sample sizes are small. By sharing information across genes, dispersion shrinkage reduces false positives caused by underestimating variability and increases the reliability of identifying differential expression.

FDR control. Testing thousands of genes inflates false positives if raw p-values are used. DESeq2 controls the FDR by adjusting p-values with the Benjamini–Hochberg method (optionally with independent hypothesis weighting [IHW]), so the expected fraction of false discoveries among significant genes stays acceptably low, e.g., < 1%.

LFC shrinkage. LFCs from low count or highly dispersed genes are unstable and often inflated. DESeq2 applies empirical Bayes shrinkage, sharing information across genes to pull noisy LFCs toward zero, yielding more precise, interpretable effect size estimates for ranking and visualization.

DESeq2 assumptions. DESeq2 first assumes that gene counts follow a negative binomial distribution, which is more effective than Poisson when dealing with high variability. It also assumes that most genes are not strongly differentially expressed, so that median-of-ratios across samples can be used to estimate size factors for normalization. Most importantly, to enable empirical Bayes shrinkage for dispersion estimation, DESeq2 assumes that genes with similar mean expression strength have similar dispersion, so that the mean–dispersion trend can be reliably fitted. It also assumes that the dispersion parameter per gene follows a log-normal prior distribution.

Functional enrichment analysis. Instead of interpreting individual genes, functional enrichment analysis determines whether predefined gene sets (e.g., Gene Ontology [GO] terms, signaling pathways, or transcription factor targets) are overrepresented in the list of significantly regulated genes. Significantly enriched categories suggest coordinated biological programs affected by the perturbation and provide mechanistic insights into the observed transcriptional changes.

Results and analysis

1. Data preprocessing

1.1 Data import and reshaping

Gene-specific read count files (.tsv) of D4 and NTC are imported into an R environment, each in triplicate, thus six files in total (controls: NTC_1.tsv, NTC_2.tsv, NTC_3.tsv; treatment: D4_1.tsv, D4_2.tsv, D4_3.tsv). Each file contains columns of gene IDs and read counts, including “num_reads_rv” (raw read counts for model fitting) and “TPM_rv” (transcripts-per-million values for zFPKM filtering). We first define the function “extract_columns” to select the columns needed for DESeq2 (“gene_id” and “num_reads_rv”) and apply it to six samples. The six tables are then combined and renamed, with repetitive gene IDs removed. Then, the “gene_id” column is converted as the row names of the count table, thus enabling the subsequent conversion from data frame to matrix. The final matrix consists of gene counts as rows and samples as columns. The following check confirms the dimensions and structure of the matrix.

```
# import
extract_columns <- function(myfile, col1, col2){
  df <- read_tsv(
    file = myfile,
    col_select = c(col1, col2)
  )
```

```
  return(df)
}

table_NTC_1 <- extract_columns("data/NTC_1.tsv", "gene_id", "num_reads_rv")
table_NTC_2 <- extract_columns("data/NTC_2.tsv", "gene_id", "num_reads_rv")
table_NTC_3 <- extract_columns("data/NTC_3.tsv", "gene_id", "num_reads_rv")
table_D4_1 <- extract_columns("data/D4_1.tsv", "gene_id", "num_reads_rv")
table_D4_2 <- extract_columns("data/D4_2.tsv", "gene_id", "num_reads_rv")
table_D4_3 <- extract_columns("data/D4_3.tsv", "gene_id", "num_reads_rv")

# reshape
# -> table
table_combined <- cbind(
  table_NTC_1, table_NTC_2, table_NTC_3, table_D4_1, table_D4_2, table_D4_3
)
count_table <- table_combined[, c(1, 2, 4, 6, 8, 10, 12)]
colnames(count_table) <- c("gene_id", "NTC_1", "NTC_2", "NTC_3", "D4_1", "D4_2", "D4_3")

# -> matrix
count_table_2 <- count_table[, c(2:7)]
rownames(count_table_2) <- count_table$gene_id
matrix_counts <- as.matrix(count_table_2)

# check
> dim(matrix_counts)
[1] 57820 6
> head(matrix_counts)
NTC_1 NTC_2 NTC_3 D4_1 D4_2 D4_3
ENSG00000223972.4 0 0 0 0 0 0
ENSG00000227232.4 549 539 493 493 491 626
ENSG00000243485.2 0 0 0 0 0 2
ENSG00000237613.2 0 0 0 0 0 0
ENSG00000268020.2 0 0 0 0 0 0
ENSG00000240361.1 0 0 0 0 0 0
```

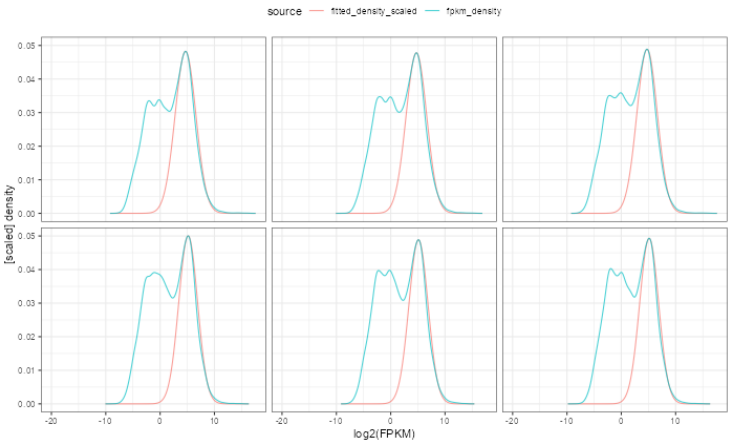
1.2 zFPKM-based filtering

We first extract the columns containing gene IDs ("gene_id") and TPM values ("TPM_rv"), and then reshape them into a table using the same steps as for the count table.

```
tpm_NTC_1 <- extract_columns("data/NTC_1.tsv", "gene_id", "TPM_rv")
tpm_NTC_2 <- extract_columns("data/NTC_2.tsv", "gene_id", "TPM_rv")
tpm_NTC_3 <- extract_columns("data/NTC_3.tsv", "gene_id", "TPM_rv")
tpm_D4_1 <- extract_columns("data/D4_1.tsv", "gene_id", "TPM_rv")
tpm_D4_2 <- extract_columns("data/D4_2.tsv", "gene_id", "TPM_rv")
tpm_D4_3 <- extract_columns("data/D4_3.tsv", "gene_id", "TPM_rv")

tpm_combined <- cbind(
  tpm_NTC_1, tpm_NTC_2, tpm_NTC_3, tpm_D4_1, tpm_D4_2, tpm_D4_3
)
tpm_table <- tpm_combined[, c(1, 2, 4, 6, 8, 10, 12)]
colnames(tpm_table) <- c("gene_id", "NTC_1", "NTC_2", "NTC_3", "D4_1", "D4_2", "D4_3")

tpm_table_2 <- tpm_table[, c(2:7)]
rownames(tpm_table_2) <- tpm_table$gene_id
```

We visually check the expression distribution of all six samples with the "zFPKMPlot" function. The output plot shows per-sample density curves of log2-scaled TPM values. For each sample, the blue curve shows a clear higher-expression peak and a lower-expression shoulder, and the red curve is fitted to the peak. The separation indicates the lowly expressed genes that need to be filtered out.

```r
library(zFPKM)

zFPKMPlot(tpm_table_2)
# -> output:
```



We then convert the TPM table to z-scores. For each gene, we extract its maximum z-score across samples and retain genes with max(z-score) > −3, and then store the corresponding gene IDs in a list (20,872 in length).

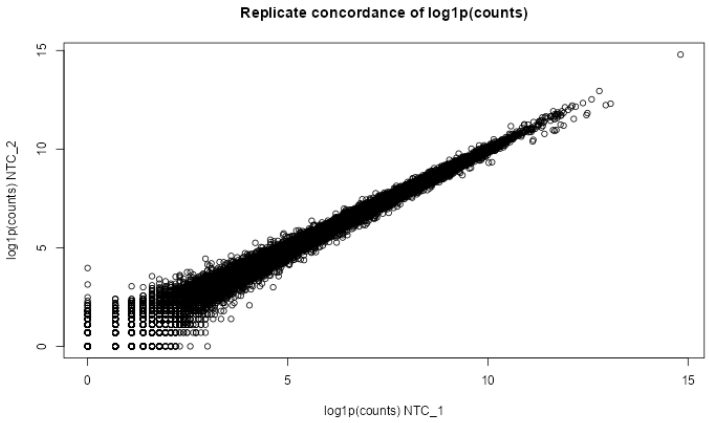
```r
z_df <- zFPKM(tpm_table_2)  # compute z-scores

max_z_df <- apply(z_df, 1, max)
selected_zTPM <- max_z_df[max_z_df > -3]  # select genes
selected_genes <- names(selected_zTPM)  # store gene IDs
length(selected_genes)  # check gene numbers
```

Finally, we use the active gene list to subset the count matrix for downstream DESeq2 analysis, then check the dimensions of the filtered count matrix (20,872 × 6).

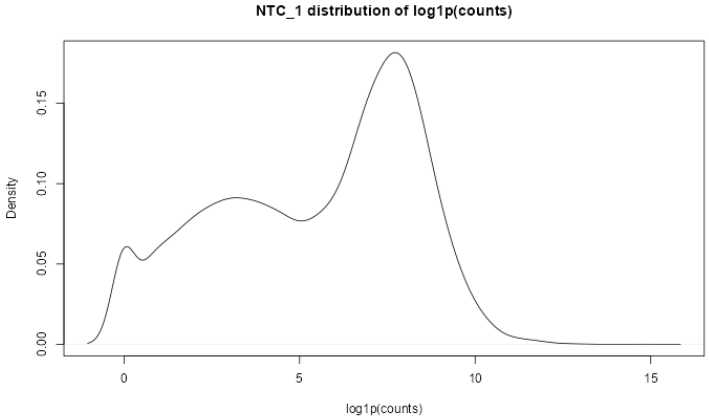
```r
matrix_counts_filtered <- matrix_counts[selected_genes, ]
dim(matrix_counts_filtered)
```

To quickly check the data, we visualize log-transformed counts using the “log1p” function. The scatter plot shows strong concordance between two control replicates. The density plot suggests that lowly expressed genes are reduced after filtering. The boxplots show that all samples have similar global count distributions.

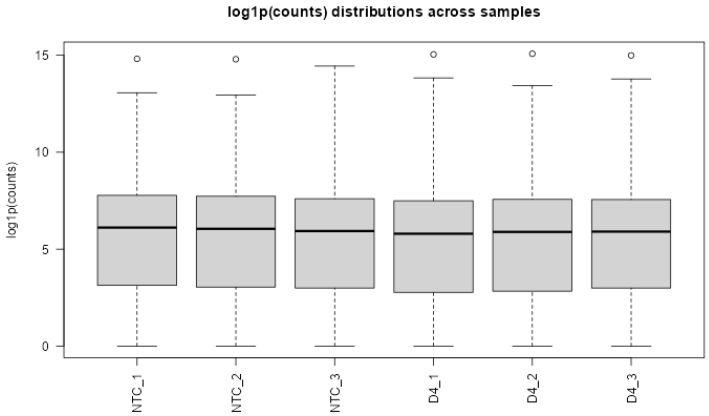
```r
plot(
  x = log1p(matrix_counts_filtered[, "NTC_1"]),
  y = log1p(matrix_counts_filtered[, "NTC_2"]),
  xlab = "log1p(counts) NTC_1",
  ylab = "log1p(counts) NTC_2",
  main = "Replicate concordance (log1p counts)"
)
# -> output:
```




```
plot(
  density(log1p(matrix_counts_filtered[, "NTC_1"])),
  main = "NTC_1 distribution of log1p(counts)",
  xlab = "log1p(counts)"
)
# -> output:
```




```
boxplot(
  log1p(matrix_counts_filtered),
  las = 2,
  main = "log1p(count distributions) across samples",
  ylab = "log1p(counts)"
)
# -> output:
```



2. Object creation

We first create a metadata (sample information) table ("colData") and rename its rows and columns to match the dataset. The columns represent experimental groups (NTC or D4) and replicate labels (1, 2, or 3), while the rows represent six samples. Releveling the "Sample" factor ensures NTC to be the reference level, i.e., DESeq2 reports LFCs as D4 compared to NTC, which is verified by the output.

```
table_colData <- data.frame(
  x = relevel(factor(c("NTC", "NTC", "NTC", "D4", "D4", "D4")), "NTC"),
  y = factor(c("1", "2", "3", "1", "2", "3"))
)
colnames(table_colData) = c("Sample", "Replicate")
rownames(table_colData) = colnames(matrix_counts)

> table_colData$Sample
[1] NTC NTC NTC D4 D4 D4
Levels: NTC D4
```

Then, we create the "DESeqDataSet" object "dds" from the filtered count matrix and the metadata table, with the design as "~ Sample" to consider only the main comparison (D4 vs NTC).

```
library(DESeq2)

# create the object
dds <- DESeqDataSetFromMatrix(
  countData = matrix_counts_filtered,
  colData = table_colData,
  design = ~ Sample
)
```

The output confirms that "dds" contains 20,872 gene counts of six samples, with their names and metadata (factors and levels) attached for model fitting.

```
> dds
class: DESeqDataSet
dim: 20872 6
metadata(1): version
assays(1): counts
rownames(20872): ENSG00000227232.4 ENSG00000237683.5 ... ENSG00000210195.2
ENSG00000210196.2
rowData names(0):
colnames(6): NTC_1 NTC_2 ... D4_2 D4_3
colData names(2): Sample Replicate
```

"dds" can also be saved as a file to be transferred among sessions.

```
# save the object as a file
dds <- qs2::qs_read("dds.qs")
```

3. Model fitting

3.1 Batch effect detection

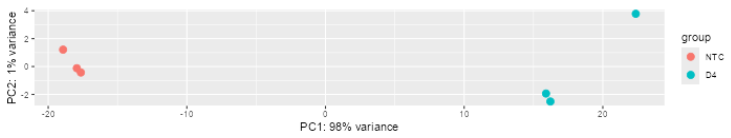
We first apply the "rlog" transformation for subsequent PCA clustering. This step minimizes differences between samples by normalizing with respect to library size for genes with low counts. The outputs show that rlog produces the "DESeqTransform" object "dds_rlog" with the same 20,872 genes and six samples. The rlog-transformed counts are stored in the object's assay ("assay(dds_rlog)"), while "rowData" contains gene-wise summaries estimated during the transformation (e.g., "baseMean" and

dispersion-related terms) and “colData” includes “Sample”, “Replicate” and the newly added “sizeFactor” representing the normalization.

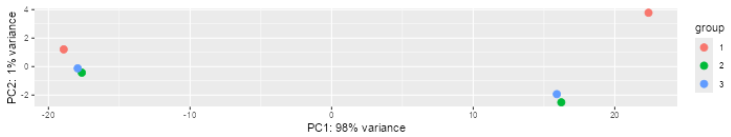
```
dds_rlog <- rlog(dds)

> dds_rlog
class: DESeqTransform
dim: 20872 6
metadata(1): version
assays(1): ''
rownames(20872): ENSG00000227232.4 ENSG00000237683.5 ... ENSG00000210195.2
ENSG00000210196.2
rowData names(7): baseMean baseVar ... dispFit rlogIntercept
colnames(6): NTC_1 NTC_2 ... D4_2 D4_3
colData names(3): Sample Replicate sizeFactor

> head(assay(dds_rlog))
NTC_1 NTC_2 NTC_3 D4_1 D4_2 D4_3
ENSG00000227232.4 8.948444 8.981718 9.003531 9.088837 9.025979 9.217338
ENSG00000237683.5 5.112067 5.271887 5.539781 5.486706 5.432516 5.627316
ENSG00000239906.1 1.246222 1.020786 1.058769 1.137262 1.060114 1.059368
ENSG00000228463.4 6.356639 6.344358 6.453789 6.266638 6.289102 6.565939
ENSG00000231709.1 3.243027 3.187245 3.159439 3.294256 3.348301 3.072706
ENSG00000225972.1 3.853543 3.755140 3.842981 4.512619 3.911318 4.759632

> rowData(dds_rlog)
DataFrame with 20872 rows and 7 columns
baseMean baseVar allZero dispGeneEst dispGeneIter dispFit rlogIntercept
<numeric> <numeric> <logical> <numeric> <numeric> <numeric> <matrix>
ENSG00000227232.4 530.47071 4122.36560 FALSE 0.0123874 7 0.0949103 9.04431
ENSG00000237683.5 43.76345 190.91870 FALSE 0.0920279 8 0.1867933 5.41171
ENSG00000239906.1 2.22400 6.38921 FALSE 0.9076634 4 2.0572774 1.09709
ENSG00000228463.4 83.90968 208.63263 FALSE 0.0153307 2 0.1388794 6.37941
ENSG00000231709.1 9.47876 14.51497 FALSE 0.0514389 36 0.5490173 3.21750
... ... ... ... ... ... ... ...
ENSG00000198695.2 3581.847 4.64958e+05 FALSE 0.0359059 8 0.0878720 11.78711
ENSG00000210194.1 222.345 1.48820e+03 FALSE 0.0292263 8 0.1063597 7.78102
ENSG00000198727.2 218022.024 1.86735e+10 FALSE 0.3928431 8 0.0866685 17.51274
ENSG00000210195.2 286.685 2.70910e+04 FALSE 0.3254361 8 0.1019359 8.00295
ENSG00000210196.2 107.290 6.40034e+02 FALSE 0.0444170 5 0.1274975 6.72174

> colData(dds_rlog)
DataFrame with 6 rows and 3 columns
Sample Replicate sizeFactor
<factor> <factor> <numeric>
NTC_1 NTC 1 1.167816
NTC_2 NTC 2 1.100982
NTC_3 NTC 3 0.980785
D4_1 D4 1 0.885408
D4_2 D4 2 0.950695
D4_3 D4 3 0.967206
```

Using dds_rlog, PCA plots show that samples cluster primarily by “Sample”, indicating that the treatment effect is strong and consistent across replicates. However, when colored by “Replicate”, there is additional clustering across PC2 (~1% variance), suggesting a potential batch effect.

```
plotPCA(dds_rlog, intgroup = "Sample")
# -> output:
```




```
plotPCA(dds_rlog, intgroup = "Replicate")
# -> output:
```

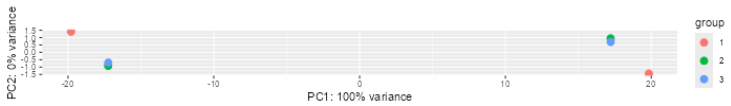


Therefore, we test whether the replicate-associated clustering can be removed (for visualization only, not for differential expression analysis). To this end, we extract the rlog-transformed count matrix from dds_rlog, set the "Replicate" factor as the batch variable, and include the "Sample" factor to preserve the biological effect. We then apply "sva::ComBat" to correct batch effect, and the batch-corrected matrix is copied into a copy of dds_rlog by simply replacing its assay for visualization.

```
matrix_rlog <- assay(dds_rlog)
batch_replicate <- colData(dds_rlog)$Replicate
model_main <- model.matrix(~ Sample, data = colData(dds))

matrix_rlog_nobatch <- sva::ComBat(
  dat = matrix_rlog,
  batch = batch_replicate,
  mod = model_main
)

dds_rlog_nobatch <- dds_rlog
assay(dds_rlog_nobatch) <- matrix_rlog_nobatch
```

After batch correction, PCA no longer shows clustering by "Replicate" (PC2 ≈ 0%), supporting the presence of a replicate-associated batch effect.

```
plotPCA(dds_rlog_nobatch, intgroup = "Replicate")
# -> output:
```



Therefore, we create a second object, "dds2", which accounts for the batch effect by including "Replicate" as a covariate in the design matrix.

```
dds2 <- DESeqDataSetFromMatrix(
  countData = assay(dds),
  colData = colData(dds),
  design = ~ Replicate + Sample  # batch first
)
```

3.2 Model fitting

Finally, we fit the DESeq2 model using the "DESeq" function, which produces the fitted "DESeqDataSet" object "dds_a". During the fitting, DESeq2 first estimates size factors to normalize

for library size, then estimates gene-wise dispersions and fits a mean count–dispersion relationship to stabilize the dispersion estimates. It fits a GLM for each gene and performs Wald tests to assess differential expression of D4 vs NTC.

```
> dds_a <- DESeq(dds2)
estimating size factors
estimating dispersions
gene-wise dispersion estimates
mean-dispersion relationship
final dispersion estimates
fitting model and testing
```

As expected, dds_a still contains 20,872 genes and six samples. It now stores multiple assays generated during the fitting, including newly added "mu" (mean counts) and "cooks" (used for detecting outliers)[1]. The "rowData" now contains gene-level results estimated by DESeq2, including dispersion-related statistics.

```
> dds_a
class: DESeqDataSet
dim: 20872 6
metadata(1): version
assays(4): counts mu H cooks
rownames(20872): ENSG00000227232.4 ENSG00000237683.5 ... ENSG00000210195.2
ENSG00000210196.2
rowData names(30): baseMean baseVar ... deviance maxCooks
colnames(6): NTC_1 NTC_2 ... D4_2 D4_3
colData names(3): Sample Replicate sizeFactor
```

It is confirmed that dds_a includes replicate contrasts relative to "Replicate_1" and the primary contrast of D4 vs NTC.

```
> resultsNames(dds_a)
[1] "Intercept" "Replicate_2_vs_1" "Replicate_3_vs_1" "Sample_D4_vs_NTC"
```

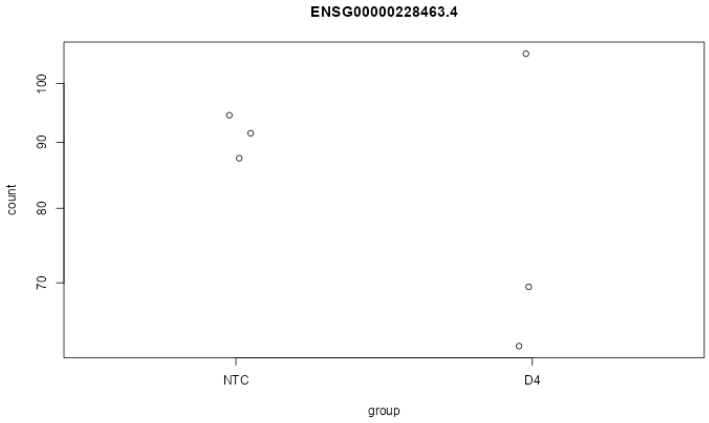
We also check the raw and size factor-normalized counts in dds_a.

```
counts_raw <- counts(dds_a)
counts_normalized <- counts(dds_a, normalized = TRUE)

> head(counts_raw)
NTC_1 NTC_2 NTC_3 D4_1 D4_2 D4_3
ENSG00000227232.4 549 539 493 493 491 626
ENSG00000237683.5 27 36 52 43 42 59
ENSG00000239906.1 8 0 1 3 1 1
ENSG00000228463.4 94 87 91 62 69 105
ENSG00000231709.1 12 9 7 11 14 4
ENSG00000225972.1 9 5 7 35 9 56

> head(counts_normalized)
NTC_1 NTC_2 NTC_3 D4_1 D4_2 D4_3
ENSG00000227232.4 470.108331 489.563058 502.658443 556.805346 516.464112 647.224961
ENSG00000237683.5 23.120082 32.698089 53.018740 48.565172 44.178193 61.000436
ENSG00000239906.1 6.850395 0.000000 1.019591 3.388268 1.051862 1.033906
ENSG00000228463.4 80.492137 79.020382 92.782796 70.024202 72.578460 108.560097
ENSG00000231709.1 10.275592 8.174522 7.137138 12.423649 14.726064 4.135623
ENSG00000225972.1 7.706694 4.541401 7.137138 39.529791 9.466756 57.898719
```

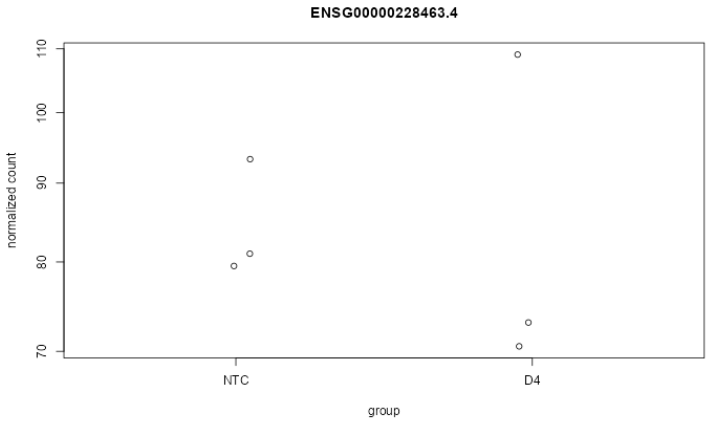
We then visually check gene-specific count patterns with the example gene "ENSG00000228463.4",

comparing raw and normalized counts and grouping the latter according by “Sample” and “Replicate”. No solid conclusions can be drawn, though.

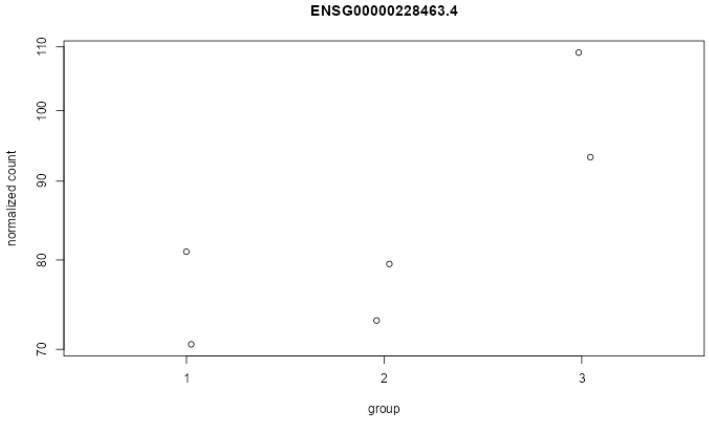
```
# raw counts
plotCounts(
  dds = dds_a,
  gene = "ENSG00000228463.4",
  intgroup = "Sample",
  normalized = FALSE
)
# ->output:
```




```
# normalized counts
plotCounts(
  dds = dds_a,
  gene = "ENSG00000228463.4",
  intgroup = "Sample",
)
# -> output:
```



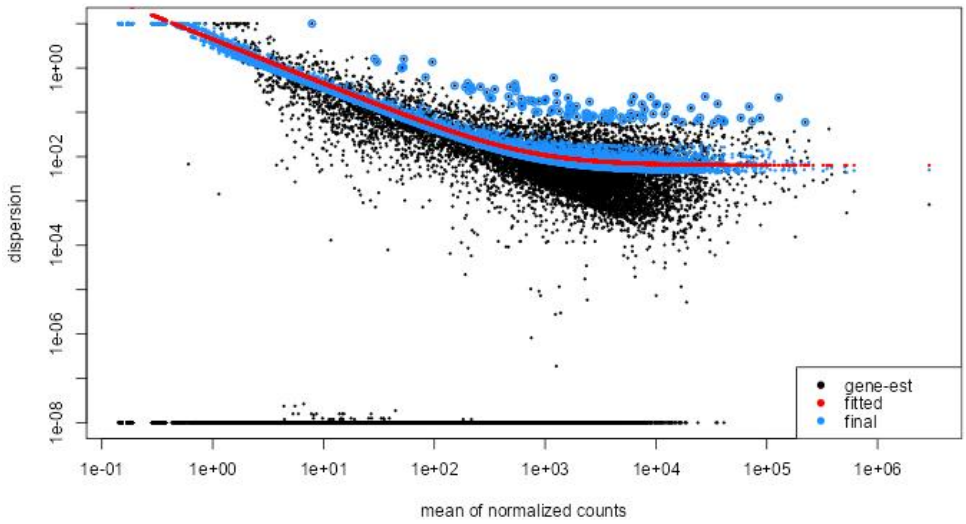

```
plotCounts(
  dds = dds_a,
  gene = "ENSG00000228463.4",
  intgroup = "Replicate",
)
# -> output:
```



3.3 Dispersion shrinkage

Importantly, we examine the dispersion plot to verify that the fitted trend is not distorted by low count outliers. The red curve captures the relationship that lowly expressed genes tend to have higher dispersion. The blue dots representing the final dispersion estimates mostly follow the red curve, which suggests that the noisy, raw estimates have been stabilized while still allowing for gene-specific variations. The blue hollow circles indicate genes whose dispersion are not shrunk.

```
plotDispEsts(dds_a)
# -> output:
```



4. Result calling

4.1 Result calling

We call differential expression results of D4 vs NTC using “results()” and control FDR using “alpha = 0.01”, which means that genes are considered significantly regulated if their adjusted p-value is below 0.01, i.e., FDR < 1%. “dds_res_simple” contains, for each gene, the LFC, standard error “lfcSE”, p-value, and adjusted p-value “padj”.

```
dds_res_simple <- results(
```

```
  object = dds_a,
  contrast = c("Sample", "D4", "NTC"),
  alpha = 0.01
)
```

```
> head(dds_res_simple)
log2 fold change (MLE): Sample D4 vs NTC
Wald test p-value: Sample D4 vs NTC
DataFrame with 6 rows and 6 columns
baseMean log2FoldChange lfcSE stat pvalue padj
<numeric> <numeric> <numeric> <numeric> <numeric> <numeric>
ENSG00000227232.4 530.47071 0.2291414 0.137481 1.6667165 0.095570784 0.15247318
ENSG00000237683.5 43.76345 0.5543051 0.394550 1.4049042 0.160049765 0.23821723
ENSG00000239906.1 2.22400 0.2578418 1.793724 0.1437467 0.885700519 NA
ENSG00000228463.4 83.90968 -0.0279426 0.282856 -0.0987873 0.921307158 0.94687809
ENSG00000231709.1 9.47876 0.1711710 0.810644 0.2111543 0.832766876 0.88179720
ENSG00000225972.1 21.04675 2.2445249 0.641671 3.4979390 0.000468868 0.00123136
```

A positive LFC indicates higher expression in D4 compared to NTC and vice versa. dds_res_simple show that in 20,872 genes tested, 20% (4238 genes) show significant upregulation at FDR < 1%, 21% (4359 genes) show significant downregulation, altogether suggesting a decent treatment effect.

```
> summary(dds_res_simple, alpha = 0.01)
out of 20872 with nonzero total read count
adjusted p-value < 0.01
LFC > 0 (up) : 4238, 20%
LFC < 0 (down) : 4359, 21%
outliers [1] : 0, 0%
low counts [2] : 2024, 9.7%
(mean count < 3)
```

We tried using IHW for FDR control, which prioritizes hypotheses of interest. However, enabling it caused the R session to crash. A plausible explanation is an incompatibility between the IHW build and Windows, since switching from Positron to RStudio did not solve the problem. We therefore proceeded with the standard workflow.

```
# dds_res_ihw <- results(
#   object = dds_a,
#   contrast = c("Sample", "D4", "NTC"),
#   filterFun = IHW::ihw,
#   alpha = 0.01
# )
```

4.2 LFC shrinkage

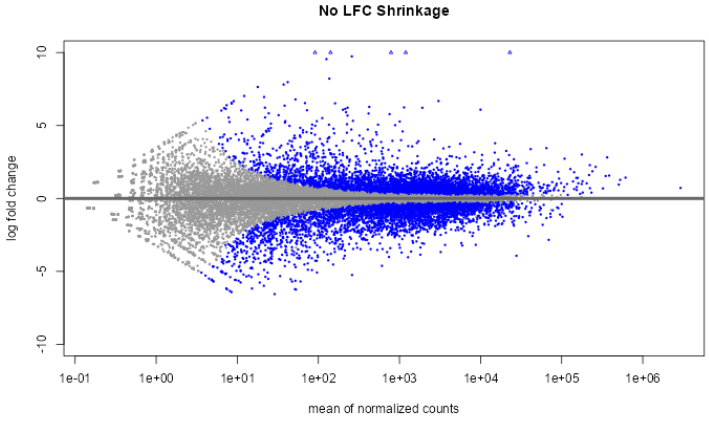
To stabilize noisy fold changes, especially for low counts, we apply and compare three LFC shrinkage methods: the default “normal”, “ashr”, and “apeglm”. For apeglm, DESeq2 requires a coefficient name, which is returned by “resultNames()” (“Sample_D4_vs_NTC”). For normal and ashr, a contrast needs to be specified instead. The results across all methods remain unchanged, because shrinkage only modifies the magnitude of LFCs and does not affect p-values and adjusted p-values.

```
# normal
dds_res_normal <- lfcShrink(
  dds = dds_a,
  contrast = c("Sample", "D4", "NTC"),  # compare D4 to NTC
  res = dds_res_simple,
  type = "normal"
)
```

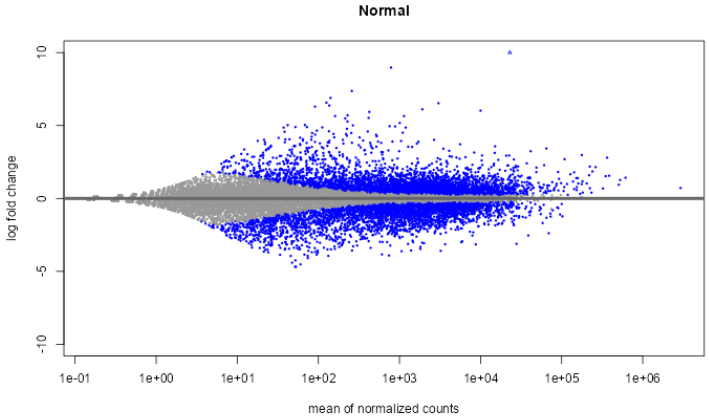
```
> summary(dds_res_normal, alpha = 0.01)
out of 20872 with nonzero total read count
adjusted p-value < 0.01
LFC > 0 (up) : 4238, 20%
LFC < 0 (down) : 4359, 21%
outliers [1] : 0, 0%
low counts [2] : 2024, 9.7%
(mean count < 3)

# ashr
dds_res_ashr <- lfcShrink(
  dds = dds_a,
  res = dds_res_simple,
  type = "ashr"
)

> summary(dds_res_ashr, alpha = 0.01)
out of 20872 with nonzero total read count
adjusted p-value < 0.01
LFC > 0 (up) : 4238, 20%
LFC < 0 (down) : 4359, 21%
outliers [1] : 0, 0%
low counts [2] : 2024, 9.7%
(mean count < 3)

# apeglm
dds_res_simple_coef <- results(
  object = dds_a,
  name = "Sample_D4_vs_NTC",
  alpha = 0.01
)

dds_res_apeglm <- lfcShrink(
  dds = dds_a,
  coef = "Sample_D4_vs_NTC",
  res = dds_res_simple_coef,
  type = "apeglm"
)

> summary(dds_res_apeglm, alpha = 0.01)
out of 20872 with nonzero total read count
adjusted p-value < 0.01
LFC > 0 (up) : 4238, 20%
LFC < 0 (down) : 4359, 21%
outliers [1] : 0, 0%
low counts [2] : 2024, 9.7%
(mean count < 3)
```

Next, we visualize the shrinkage with MA plots. Compared to the unshrunken plot, all shrunken plots show fewer dramatic LFCs in low counts, while genes with higher expression remain relatively unchanged. For downstream interpretation, we choose ashr-shrunken LFCs because it produces the most decent distribution of effect sizes.

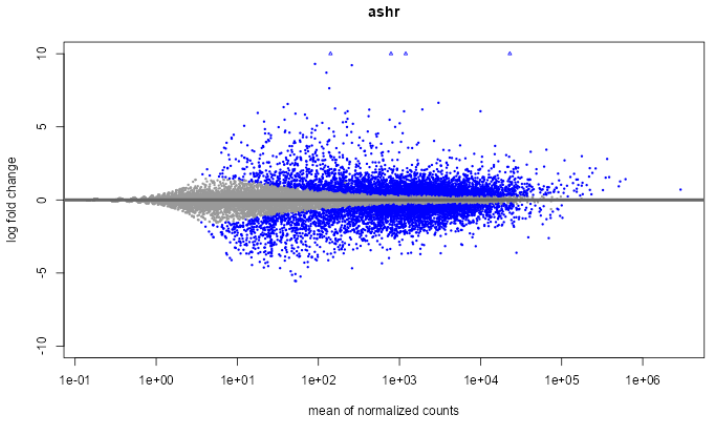
```
plotMA(
  dds_res_simple,
  ylim = c(-10, 10),
  main = "No LFC Shrinkage"
)
# -> output:
```




```
plotMA(
  dds_res_normal,
  ylim = c(-10, 10),
  main = "Normal"
)
# -> output:
```

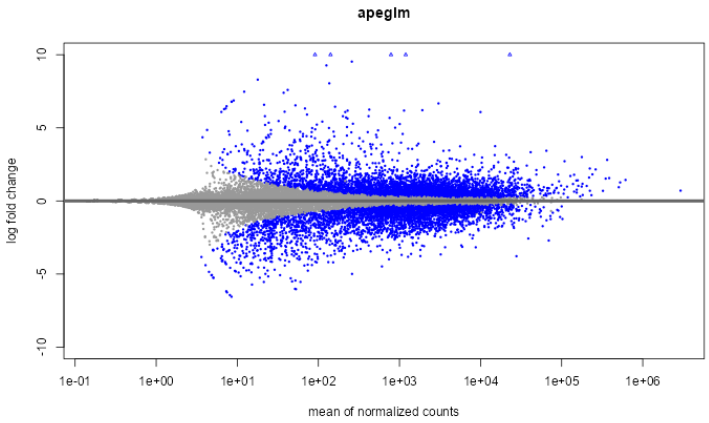



```
plotMA(
  dds_res_ashr,
  ylim = c(-10, 10),
  main = "ashr"
)
# -> output:
```




```
plotMA(
```

```
  dds_res_apeglm,
  ylim = c(-10, 10),
  main = "apeglm"
)
# -> output:
```



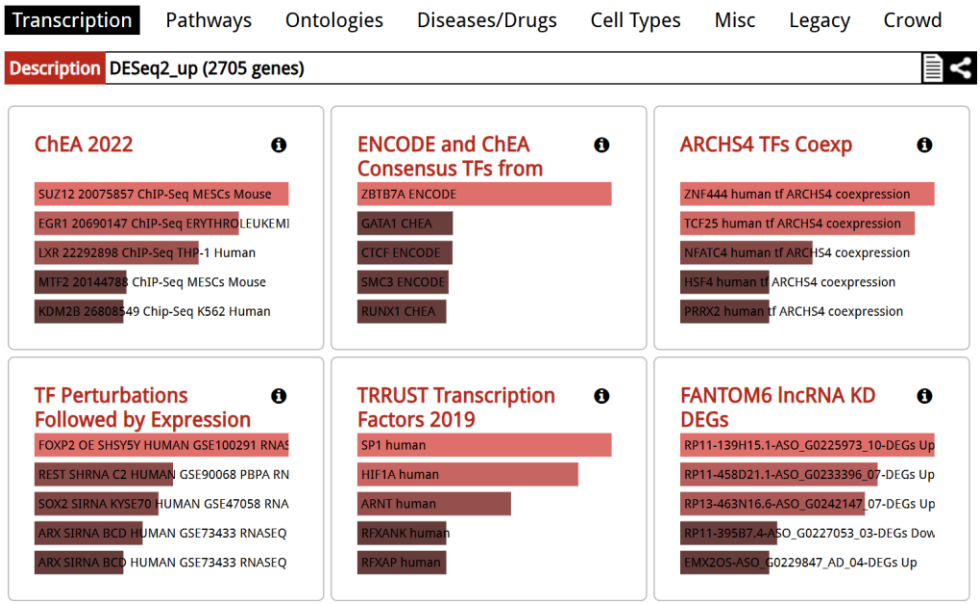
Finally, we obtain results by selecting genes with the adjusted p-value < 0.01 and LFC > 1.5, after converting "dds_res_ashr" to a data frame and removing entries with "NA". As Ensembl gene IDs are stored as row names in dds_res_ashr, we copy them into an explicable column for downstream merging with annotations. We then joined GRCh37 which also contains Ensembl gene IDs ("ensembl_gene_id_version") and corresponding names ("external_gene_name") to our result table and export it to an Excel file.

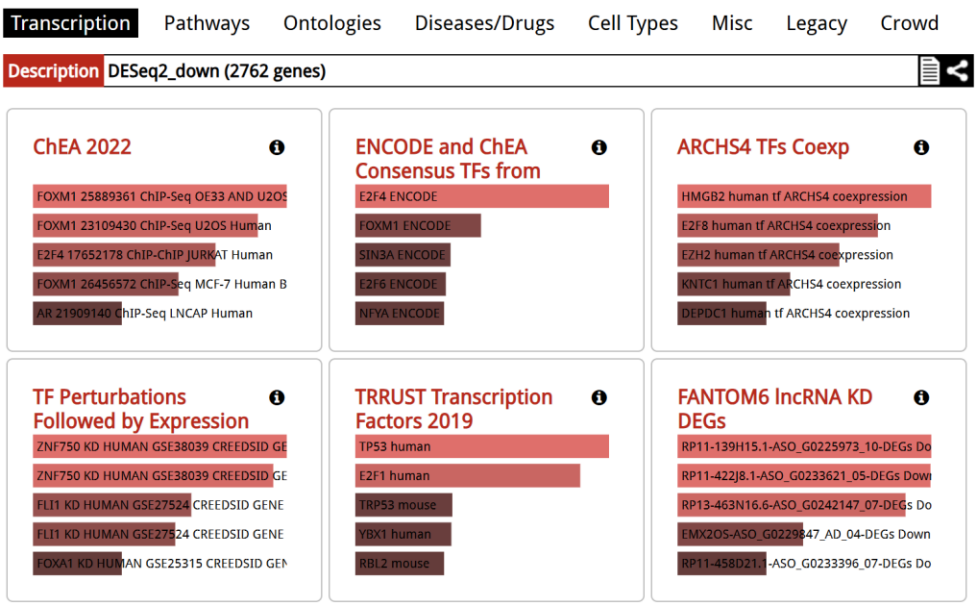
```
# select
df_dds_res_ashr <- as.data.frame(dds_res_ashr)
df_ashr_filtered <- df_dds_res_ashr[!is.na(df_dds_res_ashr$padj),]
df_ashr_filtered_2 <- df_ashr_filtered[df_ashr_filtered$padj < 0.01,]
f_ashr_filtered <- df_ashr_filtered_2[abs(df_ashr_filtered_2$log2FoldChange)
  > log2(1.5),]

# get annotations
table_annotation_all <- qs2::qd_read("data/GRCh37_annotation.qd")

> head(table_annotation_all)
ensembl_gene_id_version external_gene_name gene_biotype
1 ENSG00000261657.1 SLC25A26 protein_coding
2 ENSG00000223116.1 AL157931.1 miRNA
3 ENSG00000233440.2 HMGA1P6 pseudogene
4 ENSG00000207157.1 RNY3P4 misc_RNA
5 ENSG00000229483.2 LINC00362 lincRNA
6 ENSG00000252952.1 RNU6-58P snRNA
description
1 solute carrier family 25 (S-adenosylmethionine carrier), member 26 [Source:HGNC
Symbol;Acc:20661]
2
3 high mobility group AT-hook 1 pseudogene 6 [Source:HGNC Symbol;Acc:19121]
4 RNA, Ro-associated Y3 pseudogene 4 [Source:HGNC Symbol;Acc:42488]
5 long intergenic non-protein coding RNA 362 [Source:HGNC Symbol;Acc:42682]
6 RNA, U6 small nuclear 58, pseudogene [Source:HGNC Symbol;Acc:42548]

table_annotation <- table_annotation_all[, c("ensembl_gene_id_version",
"external_gene_name")]

> dim(table_annotation)
[1] 63677 2
```

```
# annotate
f_ashr_filtered$ensembl_gene_id_version <- rownames(f_ashr_filtered)
f_ashr_filtered <- dplyr::left_join(f_ashr_filtered, table_annotation)

> head(f_ashr_filtered)
baseMean log2FoldChange lfcSE pvalue padj ensembl_gene_id_version
1 21.04675 1.6393693 0.67833750 4.688684e-04 1.231358e-03 ENSG00000225972.1
2 3776.28896 0.8530166 0.08939757 1.635701e-22 2.326769e-21 ENSG00000225630.1
3 1787.06548 1.1751915 0.13814067 1.163369e-18 1.266004e-17 ENSG00000237973.1
4 115.37215 3.5145778 0.37125332 2.627460e-23 3.896331e-22 ENSG00000229344.1
5 16568.06942 1.4849402 0.10264512 1.578427e-48 9.690616e-47 ENSG00000248527.1
6 104.42095 3.9349519 0.39580688 4.363781e-25 7.202149e-24 ENSG00000198744.5
external_gene_name
1 MTND1P23
2 MTND2P28
3 hsa-mir-6723
4 RP5-857K21.7
5 MTATP6P1
6 RP5-857K21.11

# export
library(writexl)

write_xlsx(
  x = f_ashr_filtered,
  path = "output/results.xlsx",
  col_names = TRUE,
  format_headers = FALSE
)
```

5. Functional enrichment analysis

Functional enrichment analysis is performed in Enrichr using two gene lists, one containing 2,705 upregulated genes and the other containing 2,762 downregulated genes. Notably, transcription factor (TF)–target libraries rank high for both lists, thus interpreted in more detail according to adjusted p-values and combined scores, which are defined by Enrichr as the z-score multiplied by the log10(p-value)[4].





For genes upregulated in D4, the top enriched regulators are ZBTB7A (adjusted p value = 8.5×10^{-8}, combined score = 30.46), followed by GATA1 (0.02906, 9.42), CTCF (0.02906, 8.55), SMC3 (0.02906, 8.40), and RUNX1 (0.02906, 8.03). These results suggest that D4-upregulated genes are enriched for targets of general transcription and chromatin-associated regulators (CTCF and SMC3) as well as lineage-associated TF programs (GATA1 and RUNX1). However, this pattern shows overrepresentation of known target sets.

Index	Name	P-value	Adjusted p-value	Odds Ratio	Combined score
1	ZBTB7A ENCODE	8.171e-10	8.498e-8	1.46	30.46
2	GATA1 CHEA	0.0009879	0.02906	1.36	9.42
3	CTCF ENCODE	0.001020	0.02906	1.24	8.55
4	SMC3 ENCODE	0.001446	0.02906	1.29	8.40
5	RUNX1 CHEA	0.001758	0.02906	1.27	8.03
6	KLF4 CHEA	0.001946	0.02906	1.30	8.13
7	EZH2 ENCODE	0.001956	0.02906	1.54	9.57
8	GATA2 CHEA	0.003203	0.04125	1.32	7.59
9	SUZ12 CHEA	0.003570	0.04125	1.22	6.85
10	TRIM28 CHEA	0.009308	0.09681	1.56	7.29

For genes downregulated in D4, enrichment is dominated by strong cell cycle-associated regulators[5,6], with E2F4 showing extremely significant enrichment (adjusted p value = 1.39×10^{-85}, combined score = 1064.56) and FOXM1 also highly enriched (6.54×10^{-24}, 546.87). Other enriched regulators include SIN3A (2.35×10^{-9}, 39.25), E2F6 (3.45×10^{-7}, 24.40), and NFYA (1.87×10^{-6}, 22.37). These results support the hypothesis that D4 treatment suppresses a broad proliferation program, such as E2F/FOXM1 axis.

Index	Name	P-value	Adjusted p-value	Odds Ratio	Combined score
1	E2F4 ENCODE	1.341e-87	1.394e-85	5.32	1064.56
2	FOXM1 ENCODE	1.257e-25	6.535e-24	9.54	546.87
3	SIN3A ENCODE	6.789e-11	2.353e-9	1.68	39.25
4	E2F6 ENCODE	1.329e-8	3.454e-7	1.35	24.40
5	NFYA ENCODE	9.004e-8	0.000001873	1.38	22.37
6	TP53 CHEA	2.154e-7	0.000003734	2.05	31.46
7	NFYB ENCODE	0.000007436	0.0001105	1.25	14.75
8	E2F1 CHEA	0.00003494	0.0004543	1.45	14.93
9	SMAD4 CHEA	0.0002423	0.002800	1.48	12.36
10	FOSL2 ENCODE	0.0003440	0.003578	1.87	14.95

References

1. Love, M. I., Huber, W. & Anders, S. Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biol.* **15**, 550 (2014).
2. Umbaugh, C. S. *et al.* Selective targeting of TBXT with DARPins identifies regulatory networks and therapeutic vulnerabilities in chordoma. *Sci. Adv.* **11**, eadu2796 (2025).
3. Hart, T., Komori, H. K., LaMere, S., Podshivalova, K. & Salomon, D. R. Finding the active genes in deep RNA-seq gene expression studies. *BMC Genomics* **14**, 778 (2013).
4. Xie, Z. *et al.* Gene Set Knowledge Discovery with Enrichr. *Curr. Protoc.* **1**, e90 (2021).
5. Hsu, J. & Sage, J. Novel functions for the transcription factor E2F4 in development and disease. *Cell Cycle* **15**, 3183–3190 (2016).
6. Zona, S., Bella, L., Burton, M. J., Nestal de Moraes, G. & Lam, E. W.-F. FOXM1: An emerging master regulator of DNA damage response and genotoxic agent resistance. *Biochim. Biophys. Acta* **1839**, 1316–1322 (2014).

Supplemental information

Analyses were performed in Positron (2025.11.0-234) with R 4.5.1 on Windows. Differential expression was analyzed with DESeq2, a Bioconductor package. Package versions and platform details are reported below. Functional enrichment analysis was performed using the Enrichr website.

```
> sessionInfo()
attached base packages:
[1] stats4 stats graphics grDevices utils datasets methods base

other attached packages:
[1] writexl_1.5.4 zFPKM_1.32.0 here_1.0.2
[4] readr_2.1.6 DESeq2_1.50.2 SummarizedExperiment_1.40.0
[7] Biobase_2.70.0 MatrixGenerics_1.22.0 matrixStats_1.5.0
[10] GenomicRanges_1.62.0 Seqinfo_1.0.0 IRanges_2.44.0
[13] S4Vectors_0.48.0 BiocGenerics_0.56.0 generics_0.1.4

loaded via a namespace (and not attached):
[1] DBI_1.2.3 rlang_1.1.6 magrittr_2.0.4 compiler_4.5.1
[5] RSQLite_2.4.4 mgcv_1.9-3 png_0.1-8 systemfonts_1.3.1
[9] vctrs_0.6.5 sva_3.58.0 pkgconfig_2.0.3 crayon_1.5.3
[13] fastmap_1.2.0 backports_1.5.0 XVector_0.50.0 labeling_0.4.3
[17] tzdb_0.5.0 ragg_1.5.0 purrr_1.2.0 bit_4.6.0
```

```
[21] cachem_1.1.0 blob_1.2.4 DelayedArray_0.36.0 BiocParallel_1.44.0
[25] irlba_2.3.5.1 parallel_4.5.1 R6_2.6.1 RColorBrewer_1.1-3
[29] SQUAREM_2021.1 limma_3.66.0 genefilter_1.92.0 numDeriv_2016.8-1.1
[33] Rcpp_1.1.0 Matrix_1.7-3 splines_4.5.1 tidyselect_1.2.1
[37] abind_1.4-8 stringfish_0.17.0 codetools_0.2-20 lattice_0.22-7
[41] tibble_3.3.0 plyr_1.8.9 withr_3.0.2 KEGGREST_1.50.0
[45] S7_0.2.1 coda_0.19-4.1 survival_3.8-3 RcppParallel_5.1.11-1
[49] Biostrings_2.78.0 pillar_1.11.1 BiocManager_1.30.27 checkmate_2.3.3
[53] renv_1.1.5 vroom_1.6.6 rprojroot_2.1.1 invgamma_1.2
[57] truncnorm_1.0-9 emdbook_1.3.14 hms_1.1.4 ggplot2_4.0.1
[61] scales_1.4.0 ashr_2.2-63 xtable_1.8-4 qs2_0.1.5
[65] glue_1.8.0 tools_4.5.1 apeglm_1.32.0 annotate_1.88.0
[69] locfit_1.5-9.12 mvtnorm_1.3-3 XML_3.99-0.20 grid_4.5.1
[73] tidyr_1.3.1 bbmle_1.0.25.1 bdsmatrix_1.3-7 AnnotationDbi_1.72.0
[77] edgeR_4.8.0 nlme_3.1-168 cli_3.6.5 textshaping_1.0.4
[81] mixsqp_0.3-54 S4Arrays_1.10.0 dplyr_1.1.4 gtable_0.3.6
[85] SparseArray_1.10.2 farver_2.1.2 memoise_2.0.1 lifecycle_1.0.4
[89] httr_1.4.7 statmod_1.5.1 bit64_4.6.0-1 MASS_7.3-65
```